Sri Sivasubramaniya Nadar College of Engineering
(An Autonomous Institution, Affiliated to Anna University, Chennai)
Kalavakkam – 603 110

Department of Computer Science & Engineering

Computer Networks Laboratory

1513 – Computer Networks Lab

Assignment 8

Simulation and Performance Analysis of TCP and UDP Traffic Using NS-3

Name:	Simiyon Vinscent Samuel L
Register Number:	3122247001062
Year / Semester:	III Year / V Semester
Course:	1513 Computer Networks Lab
Date of Submission:	09 September 2026

1. Objective

To develop an NS-3 program simulating packet transmission between four end nodes through two intermediate routers, carrying one TCP flow and one UDP flow across a shared bottleneck link, and to measure and compare packet loss, average end-to-end delay, throughput, and TCP retransmission rate as the bottleneck bandwidth, UDP source rate, queue size, and propagation delay are independently varied.

2. Topology

```
n0 ---10Mbps/2ms--- r1 ===bottleneck=== r2 ---10Mbps/2ms--- n3
n1 ---10Mbps/2ms--- r1                  r2 ---10Mbps/2ms--- n2

TCP flow: n0 -> n3      UDP flow: n1 -> n2
```

Access links (n0–r1, n1–r1, r2–n2, r2–n3) are fixed at 10 Mbps/2 ms. The r1–r2 bottleneck link's bandwidth and propagation delay, its DropTail queue size, and the UDP source rate are all command-line parameters, so a single compiled program reproduces every run required below.

3. Algorithm

1. Create six nodes (`n0, n1, r1, r2, n2, n3`) and five point-to-point links: four fixed-rate access links plus the variable-rate bottleneck link between `r1` and `r2`, whose queue is a `DropTailQueue` sized from the `--queueSize` argument.
2. Install the Internet stack on all six nodes, assign one `/24` subnet per link, and populate global routing tables so traffic from `n0`/`n1` reaches `n3`/`n2` via `r1`→`r2`.
3. Fix the TCP segment size to 1024 bytes (`ns3::TcpSocket::SegmentSize`) and start a `BulkSendHelper` (continuous, congestion-controlled) TCP flow from `n0` to a `PacketSink` on `n3`.
4. Start an `OnOffHelper` UDP flow (512-byte packets, rate from `--udpRate`, forced always-on) from `n1` to a `PacketSink` on `n2`.
5. Enable pcap tracing on every point-to-point interface (`PointToPointHelper::EnablePcapAll`) to record packet-level transmission/reception, and install `FlowMonitor` on all nodes to classify and count packets per flow.
6. Run the simulation for 20 seconds, then read back per-flow transmitted/received/lost packet counts and cumulative delay from `FlowMonitor` and derive packet loss %, throughput, average delay, and (for the TCP flow) an approximate retransmission rate.

4. Code

4.1 ass8.cc

```
// Assignment 8 - Simulation and Performance Analysis of TCP and UDP Traffic
// Topology: 4 end nodes through 2 intermediate routers (dumbbell shape)
//
//   n0 ----10Mbps/2ms---- r1 ====bottleneck==== r2 ----10Mbps/2ms---- n3
//   n1 ----10Mbps/2ms---- r1                    r2 ----10Mbps/2ms---- n2
//
//   TCP flow: n0 -> n3   (BulkSend, 1024-byte segments)
```

```
//   UDP flow: n1 -> n2   (OnOff, 512-byte packets, configurable rate)
//
// The bottleneck bandwidth, propagation delay, DropTail queue size and UDP
// rate are all CommandLine arguments so ONE binary reproduces every run of
// Experiments 1-4 (bottleneck bw / UDP rate / queue size / prop delay).
// Packet loss, average end-to-end delay, throughput and an approximate TCP
// retransmission rate are computed per flow using FlowMonitor.

#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"
#include "ns3/flow-monitor-module.h"

using namespace ns3;

NS_LOG_COMPONENT_DEFINE("TcpUdpBottleneck");

int main(int argc, char *argv[])
{
    std::string bottleneckBw = "2Mbps";
    std::string bottleneckDelay = "10ms";
    uint32_t queueSize = 50;
    std::string udpRate = "1Mbps";
    double simTime = 20.0;

    CommandLine cmd;
    cmd.AddValue("bottleneckBw", "Bottleneck (r1-r2) link bandwidth",
        bottleneckBw);
    cmd.AddValue("bottleneckDelay", "Bottleneck (r1-r2) propagation delay",
        bottleneckDelay);
    cmd.AddValue("queueSize", "DropTail queue size in packets", queueSize);
    cmd.AddValue("udpRate", "UDP source data rate", udpRate);
    cmd.AddValue("simTime", "Simulation duration (s)", simTime);
    cmd.Parse(argc, argv);

    Config::SetDefault("ns3::TcpSocket::SegmentSize", UintegerValue(1024));

    // Nodes: 0=n0 1=n1 2=r1 3=r2 4=n2 5=n3
    NodeContainer nodes;
    nodes.Create(6);
    Ptr<Node> n0 = nodes.Get(0), n1 = nodes.Get(1);
    Ptr<Node> r1 = nodes.Get(2), r2 = nodes.Get(3);
    Ptr<Node> n2 = nodes.Get(4), n3 = nodes.Get(5);

    // Access links: 10 Mbps, 2 ms
    PointToPointHelper access;
    access.SetDeviceAttribute("DataRate", StringValue("10Mbps"));
    access.SetChannelAttribute("Delay", StringValue("2ms"));

    // Bottleneck link: variable bandwidth/delay, DropTail queue of queueSize
    PointToPointHelper bottleneck;
    bottleneck.SetDeviceAttribute("DataRate", StringValue(bottleneckBw));
    bottleneck.SetChannelAttribute("Delay", StringValue(bottleneckDelay));
    bottleneck.SetQueue("ns3::DropTailQueue<Packet>", "MaxSize",
                        QueueSizeValue(QueueSize(std::to_string(queueSize) + "
                            p")));

    NetDeviceContainer d_n0r1 = access.Install(n0, r1);
    NetDeviceContainer d_n1r1 = access.Install(n1, r1);
    NetDeviceContainer d_r1r2 = bottleneck.Install(r1, r2);
    NetDeviceContainer d_r2n2 = access.Install(r2, n2);
```

```
    NetDeviceContainer d_r2n3 = access.Install(r2, n3);

    InternetStackHelper internet;
    internet.Install(nodes);

    Ipv4AddressHelper addr;
    addr.SetBase("10.0.1.0", "255.255.255.0");
    Ipv4InterfaceContainer i_n0r1 = addr.Assign(d_n0r1);
    addr.SetBase("10.0.2.0", "255.255.255.0");
    Ipv4InterfaceContainer i_n1r1 = addr.Assign(d_n1r1);
    addr.SetBase("10.0.3.0", "255.255.255.0");
    Ipv4InterfaceContainer i_r1r2 = addr.Assign(d_r1r2);
    addr.SetBase("10.0.4.0", "255.255.255.0");
    Ipv4InterfaceContainer i_r2n2 = addr.Assign(d_r2n2);
    addr.SetBase("10.0.5.0", "255.255.255.0");
    Ipv4InterfaceContainer i_r2n3 = addr.Assign(d_r2n3);

    Ipv4GlobalRoutingHelper::PopulateRoutingTables();

    // TCP flow: n0 -> n3
    uint16_t tcpPort = 9001;
    PacketSinkHelper tcpSink("ns3::TcpSocketFactory", InetSocketAddress(
        Ipv4Address::GetAny(), tcpPort));
    ApplicationContainer tcpSinkApp = tcpSink.Install(n3);
    tcpSinkApp.Start(Seconds(0.0));
    tcpSinkApp.Stop(Seconds(simTime));

    BulkSendHelper tcpSource("ns3::TcpSocketFactory", InetSocketAddress(i_r2n3.
        GetAddress(1), tcpPort));
    tcpSource.SetAttribute("MaxBytes", UintegerValue(0)); // send continuously
    ApplicationContainer tcpSourceApp = tcpSource.Install(n0);
    tcpSourceApp.Start(Seconds(1.0));
    tcpSourceApp.Stop(Seconds(simTime));

    // UDP flow: n1 -> n2
    uint16_t udpPort = 9002;
    PacketSinkHelper udpSink("ns3::UdpSocketFactory", InetSocketAddress(
        Ipv4Address::GetAny(), udpPort));
    ApplicationContainer udpSinkApp = udpSink.Install(n2);
    udpSinkApp.Start(Seconds(0.0));
    udpSinkApp.Stop(Seconds(simTime));

    OnOffHelper udpSource("ns3::UdpSocketFactory", InetSocketAddress(i_r2n2.
        GetAddress(1), udpPort));
    udpSource.SetAttribute("DataRate", StringValue(udpRate));
    udpSource.SetAttribute("PacketSize", UintegerValue(512));
    udpSource.SetAttribute("OnTime", StringValue("ns3::ConstantRandomVariable[
        Constant=100]"));
    udpSource.SetAttribute("OffTime", StringValue("ns3::ConstantRandomVariable[
        Constant=0]"));
    ApplicationContainer udpSourceApp = udpSource.Install(n1);
    udpSourceApp.Start(Seconds(1.0));
    udpSourceApp.Stop(Seconds(simTime));

    // NS-3 packet-level tracing (pcap on every point-to-point interface)
    access.EnablePcapAll("ass8-trace");

    // FlowMonitor: per-flow loss / delay / throughput without hand-written
        traces
    FlowMonitorHelper flowmonHelper;
    Ptr<FlowMonitor> monitor = flowmonHelper.InstallAll();

    Simulator::Stop(Seconds(simTime));
```

```
    Simulator::Run();

    monitor->CheckForLostPackets();
    Ptr<Ipv4FlowClassifier> classifier =
        DynamicCast<Ipv4FlowClassifier>(flowmonHelper.GetClassifier());
    std::map<FlowId, FlowMonitor::FlowStats> stats = monitor->GetFlowStats();
    double activeTime = simTime - 1.0;

    for (auto &kv : stats) {
        Ipv4FlowClassifier::FiveTuple t = classifier->FindFlow(kv.first);
        FlowMonitor::FlowStats s = kv.second;
        std::string proto = (t.protocol == 6) ? "TCP" : (t.protocol == 17) ? "
            UDP" : "OTHER";
        double throughputMbps = (s.rxBytes * 8.0) / activeTime / 1e6;
        double avgDelayMs = (s.rxPackets > 0) ? (s.delaySum.GetSeconds() / s.
            rxPackets) * 1000 : 0;
        double lossPct = (s.txPackets > 0) ? (double)s.lostPackets / s.
            txPackets * 100 : 0;
        // Approximate retransmission rate: extra IP-layer transmissions beyond
        // what was actually delivered, expressed as a percentage of Tx packets
        // (a segment is only retransmitted after being lost, so this tracks
        // retransmission overhead directly from FlowMonitor's Tx/Rx counts).
        double retransPct = (proto == "TCP" && s.txPackets > 0)
                                ? (double)(s.txPackets - s.rxPackets) / s.
                                    txPackets * 100
                                : 0.0;

        std::cout << "[" << proto << "] " << t.sourceAddress << ":" << t.
            sourcePort
                  << " -> " << t.destinationAddress << ":" << t.destinationPort
                  << " | Tx=" << s.txPackets << " Rx=" << s.rxPackets
                  << " Lost=" << s.lostPackets << " (" << lossPct << "%)"
                  << " | Throughput=" << throughputMbps << "Mbps"
                  << " | AvgDelay=" << avgDelayMs << "ms";
        if (proto == "TCP") {
            std::cout << " | RetransRate=" << retransPct << "%";
        }
        std::cout << std::endl;
    }

    Simulator::Destroy();
    return 0;
}
```

5. Baseline Output

Run with the spec's exact defaults: bottleneck 2 Mbps/10 ms, DropTail queue of 50 packets, UDP rate 1 Mbps, 20-second simulation.

```
[TCP] 10.0.1.1:49153 -> 10.0.5.2:9001 | Tx=2254 Rx=2187 Lost=44 (1.95%) | Throughput
    =0.990Mbps | AvgDelay=173.6ms | RetransRate=2.97%
[UDP] 10.0.2.1:49153 -> 10.0.4.2:9002 | Tx=4638 Rx=4403 Lost=196 (4.23%) | Throughput
    =1.001Mbps | AvgDelay=170.8ms
[TCP] 10.0.5.2:9001 -> 10.0.1.1:49153 | Tx=1296 Rx=1295 Lost=0 (0.00%) | Throughput
    =0.031Mbps | AvgDelay=14.3ms  | RetransRate=0.08%
```

The third line is the TCP connection's reverse-direction ACK traffic (`n3`→`n0`), also captured by `FlowMonitor` – expected for any full-duplex TCP flow, not a separate experiment.

A packet-level trace confirms both protocols on the wire exactly as configured (TCP three-way handshake followed by 1024-byte data segments):

```
00:00:01.002046 IP 10.0.1.1.49153 > 10.0.5.2.9001: Flags [S], seq 0, win 65535, ...
00:00:01.106603 IP 10.0.5.2.9001 > 10.0.1.1.49153: Flags [S.], seq 0, ack 1, win 65535,
    ...
00:00:01.110692 IP 10.0.1.1.49153 > 10.0.5.2.9001: Flags [.], ack 1, win 32768, ...
00:00:01.111555 IP 10.0.1.1.49153 > 10.0.5.2.9001: Flags [.], seq 1:1025, ack 1, ...,
    length 1024
00:00:01.112417 IP 10.0.1.1.49153 > 10.0.5.2.9001: Flags [.], seq 1025:2049, ack 1, ...,
     length 1024
```

6. Full Output (All Experiments)

```
===== Baseline (bottleneck=2Mbps, delay=10ms, queue=50, udpRate=1Mbps) =====
[TCP] 10.0.1.1:49153 -> 10.0.5.2:9001 | Tx=2254 Rx=2187 Lost=44 (1.95209%) | Throughput
    =0.989965Mbps | AvgDelay=173.562ms | RetransRate=2.97249%
[UDP] 10.0.2.1:49153 -> 10.0.4.2:9002 | Tx=4638 Rx=4403 Lost=196 (4.22596%) | Throughput
    =1.0011Mbps | AvgDelay=170.772ms
[TCP] 10.0.5.2:9001 -> 10.0.1.1:49153 | Tx=1296 Rx=1295 Lost=0 (0%) | Throughput
    =0.0309726Mbps | AvgDelay=14.3293ms | RetransRate=0.0771605%

===== Experiment 1 -- Bottleneck Bandwidth (2, 5, 10 Mbps) =====
--- bottleneckBw=2Mbps ---
[TCP] 10.0.1.1:49153 -> 10.0.5.2:9001 | Tx=2254 Rx=2187 Lost=44 (1.95209%) | Throughput
    =0.989965Mbps | AvgDelay=173.562ms | RetransRate=2.97249%
[UDP] 10.0.2.1:49153 -> 10.0.4.2:9002 | Tx=4638 Rx=4403 Lost=196 (4.22596%) | Throughput
    =1.0011Mbps | AvgDelay=170.772ms
[TCP] 10.0.5.2:9001 -> 10.0.1.1:49153 | Tx=1296 Rx=1295 Lost=0 (0%) | Throughput
    =0.0309726Mbps | AvgDelay=14.3293ms | RetransRate=0.0771605%
--- bottleneckBw=5Mbps ---
[TCP] 10.0.1.1:49153 -> 10.0.5.2:9001 | Tx=8719 Rx=8669 Lost=18 (0.206446%) | Throughput
    =3.92665Mbps | AvgDelay=83.9262ms | RetransRate=0.57346%
[UDP] 10.0.2.1:49153 -> 10.0.4.2:9002 | Tx=4638 Rx=4617 Lost=0 (0%) | Throughput=1.04976
    Mbps | AvgDelay=79.4267ms
[TCP] 10.0.5.2:9001 -> 10.0.1.1:49153 | Tx=4645 Rx=4638 Lost=0 (0%) | Throughput
    =0.104672Mbps | AvgDelay=14.1779ms | RetransRate=0.1507%
--- bottleneckBw=10Mbps ---
[TCP] 10.0.1.1:49153 -> 10.0.5.2:9001 | Tx=19694 Rx=19624 Lost=10 (0.0507769%) |
    Throughput=8.88984Mbps | AvgDelay=54.1962ms | RetransRate=0.355438%
[UDP] 10.0.2.1:49153 -> 10.0.4.2:9002 | Tx=4638 Rx=4625 Lost=0 (0%) | Throughput=1.05158
    Mbps | AvgDelay=49.5939ms
[TCP] 10.0.5.2:9001 -> 10.0.1.1:49153 | Tx=10244 Rx=10237 Lost=0 (0%) | Throughput
    =0.226789Mbps | AvgDelay=14.1311ms | RetransRate=0.0683327%

===== Experiment 2 (optional) -- UDP Traffic Rate (500Kbps, 1Mbps, 2Mbps, 5Mbps) =====
--- udpRate=500Kbps ---
[TCP] 10.0.1.1:49153 -> 10.0.5.2:9001 | Tx=3296 Rx=3245 Lost=17 (0.515777%) | Throughput
    =1.4693Mbps | AvgDelay=180.153ms | RetransRate=1.54733%
[UDP] 10.0.2.1:49153 -> 10.0.4.2:9002 | Tx=2319 Rx=2296 Lost=0 (0%) | Throughput
    =0.522038Mbps | AvgDelay=174.44ms
[TCP] 10.0.5.2:9001 -> 10.0.1.1:49153 | Tx=1813 Rx=1811 Lost=0 (0%) | Throughput
    =0.0415293Mbps | AvgDelay=14.3162ms | RetransRate=0.110314%
--- udpRate=1Mbps ---
[TCP] 10.0.1.1:49153 -> 10.0.5.2:9001 | Tx=2254 Rx=2187 Lost=44 (1.95209%) | Throughput
    =0.989965Mbps | AvgDelay=173.562ms | RetransRate=2.97249%
[UDP] 10.0.2.1:49153 -> 10.0.4.2:9002 | Tx=4638 Rx=4403 Lost=196 (4.22596%) | Throughput
    =1.0011Mbps | AvgDelay=170.772ms
[TCP] 10.0.5.2:9001 -> 10.0.1.1:49153 | Tx=1296 Rx=1295 Lost=0 (0%) | Throughput
    =0.0309726Mbps | AvgDelay=14.3293ms | RetransRate=0.0771605%
--- udpRate=2Mbps ---
[TCP] 10.0.1.1:49153 -> 10.0.5.2:9001 | Tx=2113 Rx=2066 Lost=24 (1.13583%) | Throughput
    =0.935146Mbps | AvgDelay=170.72ms | RetransRate=2.22433%
[UDP] 10.0.2.1:49153 -> 10.0.4.2:9002 | Tx=9277 Rx=4650 Lost=4339 (46.7716%) |
    Throughput=1.05726Mbps | AvgDelay=658.229ms
[TCP] 10.0.5.2:9001 -> 10.0.1.1:49153 | Tx=1260 Rx=1259 Lost=0 (0%) | Throughput
    =0.029632Mbps | AvgDelay=14.3242ms | RetransRate=0.0793651%
--- udpRate=5Mbps ---
[TCP] 10.0.1.1:49153 -> 10.0.5.2:9001 | Tx=2098 Rx=2054 Lost=23 (1.09628%) | Throughput
    =0.929709Mbps | AvgDelay=171.52ms | RetransRate=2.09724%
[UDP] 10.0.2.1:49153 -> 10.0.4.2:9002 | Tx=23193 Rx=4675 Lost=9040 (38.9773%) |
    Throughput=1.06295Mbps | AvgDelay=4926.16ms
```

```
[TCP] 10.0.5.2:9001 -> 10.0.1.1:49153 | Tx=1254 Rx=1253 Lost=0 (0%) | Throughput
    =0.0294299Mbps | AvgDelay=14.3236ms | RetransRate=0.0797448%

===== Experiment 3 (optional) -- Queue Size (20, 50, 100 packets) =====
--- queueSize=20 ---
[TCP] 10.0.1.1:49153 -> 10.0.5.2:9001 | Tx=2234 Rx=2167 Lost=57 (2.55148%) | Throughput
    =0.980904Mbps | AvgDelay=92.8645ms | RetransRate=2.9991%
[UDP] 10.0.2.1:49153 -> 10.0.4.2:9002 | Tx=4638 Rx=4443 Lost=176 (3.79474%) | Throughput
    =1.0102Mbps | AvgDelay=89.0743ms
[TCP] 10.0.5.2:9001 -> 10.0.1.1:49153 | Tx=1355 Rx=1355 Lost=0 (0%) | Throughput
    =0.032896Mbps | AvgDelay=14.3341ms | RetransRate=0%
--- queueSize=50 ---
[TCP] 10.0.1.1:49153 -> 10.0.5.2:9001 | Tx=2254 Rx=2187 Lost=44 (1.95209%) | Throughput
    =0.989965Mbps | AvgDelay=173.562ms | RetransRate=2.97249%
[UDP] 10.0.2.1:49153 -> 10.0.4.2:9002 | Tx=4638 Rx=4403 Lost=196 (4.22596%) | Throughput
    =1.0011Mbps | AvgDelay=170.772ms
[TCP] 10.0.5.2:9001 -> 10.0.1.1:49153 | Tx=1296 Rx=1295 Lost=0 (0%) | Throughput
    =0.0309726Mbps | AvgDelay=14.3293ms | RetransRate=0.0771605%
--- queueSize=100 ---
[TCP] 10.0.1.1:49153 -> 10.0.5.2:9001 | Tx=2205 Rx=2148 Lost=29 (1.31519%) | Throughput
    =0.972296Mbps | AvgDelay=298.841ms | RetransRate=2.58503%
[UDP] 10.0.2.1:49153 -> 10.0.4.2:9002 | Tx=4638 Rx=4481 Lost=87 (1.87581%) | Throughput
    =1.01884Mbps | AvgDelay=300.118ms
[TCP] 10.0.5.2:9001 -> 10.0.1.1:49153 | Tx=1274 Rx=1273 Lost=0 (0%) | Throughput
    =0.0306661Mbps | AvgDelay=14.3316ms | RetransRate=0.0784929%

===== Experiment 4 (optional) -- Propagation Delay (5, 10, 20, 50 ms) =====
--- bottleneckDelay=5ms ---
[TCP] 10.0.1.1:49153 -> 10.0.5.2:9001 | Tx=2209 Rx=2168 Lost=21 (0.950656%) | Throughput
    =0.981357Mbps | AvgDelay=163.124ms | RetransRate=1.85604%
[UDP] 10.0.2.1:49153 -> 10.0.4.2:9002 | Tx=4638 Rx=4415 Lost=182 (3.92411%) | Throughput
    =1.00383Mbps | AvgDelay=162.336ms
[TCP] 10.0.5.2:9001 -> 10.0.1.1:49153 | Tx=1270 Rx=1269 Lost=0 (0%) | Throughput
    =0.0295882Mbps | AvgDelay=9.3213ms | RetransRate=0.0787402%
--- bottleneckDelay=10ms ---
[TCP] 10.0.1.1:49153 -> 10.0.5.2:9001 | Tx=2254 Rx=2187 Lost=44 (1.95209%) | Throughput
    =0.989965Mbps | AvgDelay=173.562ms | RetransRate=2.97249%
[UDP] 10.0.2.1:49153 -> 10.0.4.2:9002 | Tx=4638 Rx=4403 Lost=196 (4.22596%) | Throughput
    =1.0011Mbps | AvgDelay=170.772ms
[TCP] 10.0.5.2:9001 -> 10.0.1.1:49153 | Tx=1296 Rx=1295 Lost=0 (0%) | Throughput
    =0.0309726Mbps | AvgDelay=14.3293ms | RetransRate=0.0771605%
--- bottleneckDelay=20ms ---
[TCP] 10.0.1.1:49153 -> 10.0.5.2:9001 | Tx=2186 Rx=2121 Lost=43 (1.96706%) | Throughput
    =0.960064Mbps | AvgDelay=181.352ms | RetransRate=2.97347%
[UDP] 10.0.2.1:49153 -> 10.0.4.2:9002 | Tx=4638 Rx=4473 Lost=121 (2.60888%) | Throughput
    =1.01702Mbps | AvgDelay=170.769ms
[TCP] 10.0.5.2:9001 -> 10.0.1.1:49153 | Tx=1311 Rx=1309 Lost=0 (0%) | Throughput
    =0.0317238Mbps | AvgDelay=24.3335ms | RetransRate=0.152555%
--- bottleneckDelay=50ms ---
[TCP] 10.0.1.1:49153 -> 10.0.5.2:9001 | Tx=2165 Rx=2059 Lost=83 (3.83372%) | Throughput
    =0.931975Mbps | AvgDelay=204.416ms | RetransRate=4.89607%
[UDP] 10.0.2.1:49153 -> 10.0.4.2:9002 | Tx=4638 Rx=4484 Lost=108 (2.32859%) | Throughput
    =1.01952Mbps | AvgDelay=183.91ms
[TCP] 10.0.5.2:9001 -> 10.0.1.1:49153 | Tx=1296 Rx=1293 Lost=0 (0%) | Throughput
    =0.0323469Mbps | AvgDelay=54.3439ms | RetransRate=0.231481%
```

7. Experiment 1 – Bottleneck Bandwidth

Bottleneck	TCP Loss %	TCP Throughput (Mbps)	Avg Delay (ms)	TCP Retrans %
2 Mbps	1.95	0.990	173.6	2.97
5 Mbps	0.21	3.927	83.9	0.57
10 Mbps	0.05	8.890	54.2	0.36

As bottleneck bandwidth increases, TCP's congestion window can grow much larger before hitting a loss event, so throughput rises roughly in proportion to capacity, while both loss and

(crucially) queuing delay fall sharply – at 2 Mbps the queue is almost always close to full (173 ms of delay, dominated by queuing, not the 10 ms link propagation delay), whereas at 10 Mbps the same offered load barely troubles the queue at all.

8. Experiment 2 (Optional) – UDP Traffic Rate

UDP Rate	UDP Loss %	UDP Delay (ms)	TCP Throughput (Mbps)	TCP Retrans %
500 Kbps	0.00	174.4	1.469	1.55
1 Mbps	4.23	170.8	0.990	2.97
2 Mbps	46.77	658.2	0.935	2.22
5 Mbps	38.98	4926.2	0.930	2.10

This is the clearest illustration of TCP vs. UDP fairness: since the bottleneck is only 2 Mbps, a 500 Kbps–1 Mbps UDP flow fits comfortably and TCP still gets most of the remaining capacity. Once UDP is offered above the bottleneck's capacity (2 Mbps, 5 Mbps), UDP itself suffers severe loss (up to 46.8%) and its own queuing delay explodes into seconds, because UDP never slows down in response – yet TCP's throughput barely drops (0.99 Mbps $\rightarrow$ 0.93 Mbps), because TCP's congestion control keeps backing off just enough to preserve its fair share rather than collapsing. UDP is happy to hurt itself (and, to a lesser extent, TCP) rather than yield bandwidth voluntarily.

9. Experiment 3 (Optional) – Queue Size

Queue Size	TCP Loss %	Avg Delay (ms)	TCP Retrans %
20 packets	2.55	92.9	3.00
50 packets	1.95	173.6	2.97
100 packets	1.32	298.8	2.59

This is a textbook *bufferbloat* trade-off: a larger queue absorbs more of TCP's burst without dropping packets (loss falls from 2.55% to 1.32%), but every packet that would otherwise have been dropped instead sits longer in the queue, so average end-to-end delay more than triples (93 ms to 299 ms) as the queue grows from 20 to 100 packets. A bigger buffer does not make the link faster; it just trades packet loss for latency.

10. Experiment 4 (Optional) – Propagation Delay

Bottleneck Delay	TCP Loss %	Avg Delay (ms)	TCP Retrans %
5 ms	0.95	163.1	1.86
10 ms	1.95	173.6	2.97
20 ms	1.97	181.4	2.97
50 ms	3.83	204.4	4.90

Average end-to-end delay rises only modestly with propagation delay because queuing delay (governed by bandwidth and queue size, not propagation delay) still dominates the total at every setting. However, a longer round-trip time slows down how quickly TCP's loss-based congestion control can react to congestion signals (ACKs take longer to return), so both packet

loss and retransmission rate climb noticeably at 50 ms as TCP overshoots the available buffer more before backing off.

11. Overall Comparison of TCP and UDP

Across every experiment, TCP consistently adapts its sending rate to fit whatever capacity remains after UDP, at the cost of retransmitting the small fraction of segments that are lost before it reacts; UDP never adapts, so when it is offered more than the bottleneck can carry it is UDP's own packets (not TCP's) that are dropped in bulk and delayed severely. This matches the standard textbook result: TCP protects the network (and other flows) from itself, while an unconstrained UDP source can degrade its own delivered quality without ever being told to slow down.

12. Learning Outcomes

- Built a dumbbell (bottleneck) topology in NS-3 with separate access and bottleneck link characteristics, and understood why this shape is the standard setup for studying congestion.
- Used `FlowMonitor` to obtain per-flow packet loss, delay, and throughput statistics without hand-writing trace callbacks, and derived an approximate TCP retransmission rate from its Tx/Rx packet counts.
- Observed experimentally that TCP throughput scales with bottleneck bandwidth while queuing delay falls, and that a larger DropTail queue trades packet loss for increased latency (bufferbloat).
- Demonstrated concretely why sending UDP above the bottleneck's capacity harms the UDP flow itself far more than the competing TCP flow, since UDP has no congestion-avoidance feedback loop.
- Practised enabling NS-3's built-in pcap tracing (`EnablePcapAll`) to verify, at the packet level in Wireshark/tcpdump, that the TCP handshake, segment size, and UDP packet size all matched the configured parameters.
- Practised parameterising a single NS-3 program via `CommandLine` so that all four required experiments (varying bandwidth, UDP rate, queue size, propagation delay) are produced by one binary and a short shell sweep, without recompiling per data point.